

BloodLink v1.0 — Phase 7: Release Plan

Field	Detail
Project	BloodLink — Real-time Blood Bank Management & Intelligent Donor Matching System
Version	1.0 (Production Release)
Date	May 2026
Prepared By	Project Team (Team 3)
Status	Ready for Release

1. Release Plan (v1.0)

1.1 Release Overview

BloodLink v1.0 is the first production release of the real-time Blood Bank Management and Intelligent Donor Matching System. This release delivers a fully functional clinical platform built with **NestJS 11** (backend), **Next.js 16** (frontend), and **PostgreSQL 17** (database), enabling hospitals across Cairo to manage blood inventory, coordinate donor networks, process emergency blood requests, and leverage an intelligent multi-factor matching algorithm for optimal donor-patient pairing.

1.2 Release Scope

Module	Features Included	Status
Authentication	JWT-based login, bcrypt hashing, role-based access (Admin/Staff/Manager)	 Complete
Blood Inventory	Real-time tracking of 8 blood types, auto-status thresholds, expiry alerts	 Complete
Donor Management	Full CRUD, search/filter, availability toggle, eligibility tracking	 Complete
Emergency Requests	Create/edit/delete emergencies, 4 urgency levels, hospital tracking	 Complete
Matching Engine v2	Multi-factor scoring (reliability, proximity, exact-match, recency penalty)	 Complete
Notifications	5 categories, mark-as-read, dismiss, bulk actions, unread badge	 Complete
User Profiles	View/edit profile, hospital assignment	 Complete

1.3 Release Timeline

Phase	Date	Milestone
Code Freeze	May 1, 2026	All feature branches merged to <code>main</code>
Phase 6 — Testing Complete	May 5, 2026	53 unit tests + 28 integration + 4 E2E passing
Bug Fixes & Regression	May 5–7, 2026	All Critical/High bugs resolved
Phase 7 — Release Prep	May 7, 2026	Release plan, KPIs, risk assessment finalized
UAT Sign-off	May 10, 2026	Stakeholder acceptance testing

1.4 Release Checklist

#	Task	Owner	Status
1	All 85 automated tests pass (53 unit + 28 integration + 4 E2E)	Mohamed Ward	✓
2	Zero Critical/High severity bugs open	Mohamed Ward	✓
3	Environment variables documented in <code>.env.example</code>	Yousef Samy	✓
4	Database seed script verified for first-launch initialization	Yousef Samy	✓
5	README.md updated with full API reference and setup guide	Hamdy Ahmed	✓
6	Git tag <code>v1.0</code> created on <code>main</code> branch	Hamdy Ahmed	⌚ Pending
7	UAT sign-off obtained from stakeholders	Hamdy Ahmed	⌚ Pending

2. Resource Allocation Table

2.1 Team Members & Role Allocation

#	Team Member	Primary Role	Secondary Role	Allocation (%)	Phase 7 Responsibilities
1	Hamdy Ahmed (Team Leader)	Project Manager & Lead Developer	Frontend / Backend Development	100%	Release coordination, full-stack development, code review, stakeholder sign-off
2	Mohamed Abdelrazek	Frontend Developer	Database Administrator	85%	Next.js UI development, UI/UX polish, Cypress E2E maintenance, bug fixes
3	Yousef Samy	Backend Developer	Documentation Lead	80%	NestJS API development, database schema management, README & release notes
4	Mohamed Ward	QA Lead	Backend Support	75%	Final regression testing, test report compilation, defect triage, release verification
5	Ibrahim Abdelkader	Testing & Integration	Frontend Support	70%	Integration testing, evidence collection, CI/CD pipeline, quality assurance

2.2 Infrastructure Resources

Resource	Specification	Purpose
Application Server	Node.js 18+, 4 vCPU, 8GB RAM	Hosts NestJS backend (port 3001)
Web Server	Node.js 18+, 2 vCPU, 4GB RAM	Hosts Next.js frontend (port 3000)
Database Server	PostgreSQL 17, 4 vCPU, 16GB RAM, 100GB SSD	Production database with daily backups
CI/CD Pipeline	GitHub Actions (.github/workflows/test.yml)	Automated testing on every push/PR
Version Control	GitHub Repository (hamdy-fathi/BloodLink)	Source code management, branching strategy

2.3 Time Allocation by Activity

Activity	Estimated Hours	Assigned To
Backend API finalization & hardening	8 hrs	Yousef Samy, Hamdy Ahmed
Frontend UI/UX final polish & optimization	8 hrs	Mohamed Abdelrazek, Hamdy Ahmed
Final regression test suite execution	6 hrs	Mohamed Ward, Ibrahim Abdelkader
Database seed verification & schema validation	4 hrs	Yousef Samy
Documentation & release notes	4 hrs	Hamdy Ahmed, Ibrahim Abdelkader
UAT support & stakeholder demos	6 hrs	Hamdy Ahmed
Total	36 hrs	

3. Key Performance Indicators (KPIs)

KPI 1 — System Uptime (SMART)

SMART Criteria	Definition
Specific	Maintain BloodLink system availability (backend API + frontend + database) at or above the target uptime percentage
Measurable	Monitored via automated health-check endpoint (GET /api/health) pinged every 60 seconds; uptime = (total_pings – failed_pings) / total_pings × 100
Achievable	Industry standard for clinical systems is 99.5%; target of 99.0% is conservative and achievable with single-server deployment
Relevant	Blood bank operations are time-critical; system downtime during emergencies can directly impact patient outcomes and donor coordination
Time-bound	Measured over the first 30 days post-deployment (May 12 – June 11, 2026)

Target: ≥ 99.0% system uptime over the first 30 days of production operation.

Measurement: Automated health-check endpoint pinged every 60 seconds. Uptime = (successful responses / total pings) × 100.

Current Baseline: Development environment achieved 99.8% uptime during the 4-week testing phase.

KPI 2 — Emergency Response Time (SMART)

SMART Criteria	Definition
Specific	Reduce the average time from emergency blood request creation to matched donor notification to under 30 seconds
Measurable	Measured as the elapsed time (in seconds) between <code>POST /api/emergencies</code> and completion of <code>POST /api/emergencies/:id/notify</code> , tracked via server-side timestamps in the application logs
Achievable	Current matching engine v2 processes and scores all compatible donors within 2–5 seconds in testing; 30-second target includes user interaction time for review and confirmation
Relevant	In critical blood emergencies, every minute counts; faster donor matching and notification directly improves patient survival rates
Time-bound	Achieve and maintain this target within 60 days of go-live (May 12 – July 11, 2026)

Target: Average emergency-to-notification time ≤ 30 seconds for Critical and High urgency requests.

Measurement: Server-side timestamp delta between emergency creation and donor notification dispatch. Logged per transaction and aggregated weekly.

Current Baseline: Testing phase average = 8 seconds (automated) / 22 seconds (including manual review).

KPI 3 — Defect Escape Rate

Target: ≤ 5 production defects reported by end users in the first 60 days post-release.

Measurement: Count of new bug reports filed with severity High or Critical after v1.0 deployment. Tracked via the project's bug tracking system using the `BL-BUG-XXX` identifier format.

Rationale: Phase 6 testing identified and resolved 8 defects (2 Critical, 2 High, 2 Medium, 2 Low) across 85 automated tests. A low defect escape rate validates the effectiveness of the QA process and ensures production stability for clinical operations. This KPI directly reflects code quality and testing coverage adequacy.

Current Baseline: 0 known Critical/High defects at time of release; 85 automated tests providing coverage across all 5 backend modules and 7 frontend pages.

4. Risk Management — Top 3 Risks

Risk 1: Database Failure or Data Loss

Attribute	Detail
Risk ID	R-001
Category	Technical / Infrastructure
Description	PostgreSQL database server experiences hardware failure, corruption, or accidental data deletion, resulting in loss of critical blood inventory records, donor data, or emergency request history
Probability	Medium (2/5)
Impact	● Critical (5/5) — Loss of blood inventory data could lead to incorrect stock levels and endanger patient safety
Risk Score	10 / 25 (High)
Mitigation Strategies	1. Automated daily backups using <code>pg_dump</code> scheduled via cron job at 02:00 AM, retained for 30 days
	2. Point-in-time recovery (PITR) enabled via PostgreSQL WAL archiving for granular data restoration
	3. Pre-deployment database snapshot taken before every release and stored offsite
	4. TypeORM migration scripts maintained to allow schema recreation from scratch if needed
Contingency Plan	Restore from the most recent backup; maximum acceptable data loss (RPO) = 24 hours. Notify all hospital stakeholders immediately if data loss exceeds 1 hour
Owner	Yousef Samy (Backend Developer)
Status	Mitigation in progress — backup scripts under development

Risk 2: Security Breach via JWT Token Compromise

Attribute	Detail
Risk ID	R-002
Category	Security
Description	An attacker intercepts or brute-forces a JWT token, gaining unauthorized access to the system. Since tokens are stored in localStorage and the JWT_SECRET is configured via environment variables, a weak secret or XSS vulnerability could expose patient and donor data
Probability	Low (2/5)
Impact	● Critical (5/5) — Unauthorized access to donor personal information, blood inventory manipulation, or fake emergency creation could have severe clinical and legal consequences
Risk Score	10 / 25 (High)
Mitigation Strategies	1. Strong JWT_SECRET — minimum 256-bit random key generated via <code>crypto.randomBytes(64).toString('hex')</code> 2. Token expiration — JWT tokens expire after 8 hours, forcing re-authentication 3. HTTPS enforcement — all production traffic served over TLS to prevent token interception 4. Input validation — <code>class-validator</code> with <code>forbidNonWhitelisted: true</code> rejects all unexpected payload fields 5. CORS restriction — only whitelisted production domain allowed
Contingency Plan	Rotate JWT_SECRET immediately upon suspicion of compromise; invalidate all active sessions; conduct security audit within 48 hours
Owner	Yousef Samy + Hamdy Ahmed
Status	Partially mitigated — JWT expiry and validation in place; HTTPS and secret rotation SOP pending

Risk 3: Matching Engine Produces Incorrect Donor Rankings

Attribute	Detail
Risk ID	R-003
Category	Clinical / Algorithmic
Description	The Matching Engine v2 returns incorrectly scored or ranked donors due to floating-point precision errors, incorrect blood compatibility logic, or stale GPS coordinate data, potentially matching an incompatible donor to a patient emergency
Probability	Low (1/5)
Impact	 Critical (5/5) — An incorrect blood type match could result in a transfusion reaction, posing a direct threat to patient life
Risk Score	5 / 25 (Medium)
Mitigation Strategies	1. Comprehensive unit tests — 20+ tests covering Haversine calculation, scoring formula, weight profiles, recency penalty, and boundary conditions
	2. Blood compatibility matrix validated against WHO medical standards for all 8 ABO/Rh type combinations
	3. Score clamping enforced (0–100 range) with <code>Math.min(100, Math.max(0, score))</code> to prevent anomalous values
	4. Human review step — matched donors are displayed for staff review before notification; no fully automated transfusion decisions
	5. Exact-match bonus clearly flagged in UI so staff can visually confirm blood type compatibility
Contingency Plan	If a matching error is detected: immediately suspend the matching engine, revert to manual donor selection, and hotfix within 24 hours. Post-incident review required within 72 hours
Owner	Yousef Samy + Mohamed Ward
Status	Fully mitigated — all unit tests passing; manual review step enforced in workflow

Risk Summary Matrix

Risk ID	Risk Description	Probability	Impact	Score	Priority
R-001	Database failure or data loss	Medium	Critical	10/25	● High
R-002	JWT token security breach	Low	Critical	10/25	● High
R-003	Matching engine incorrect results	Low	Critical	5/25	● Medium

5. Approval

Name	Role	Signature	Date
	Project Manager		
	QA Lead		
	Tech Lead		
	Stakeholder		
